

Description

Entity-Relationship (ER) diagram for the TravelGo platform:

Entities Involved: The diagram consists of two primary entities:

Users: Represents individuals using the TravelGo platform.

Bookings: Represents individual travel or accommodation reservations made by users.

Primary Keys:

Users: `email` (PK) uniquely identifies each user, serving as their unique identifier.

Bookings: `booking_id` (PK) uniquely identifies each specific booking record.

Relationships:

Users to Bookings: There is a "1 to Many" relationship between `Users` and `Bookings`. This indicates that one user can make multiple bookings, but each booking is associated with only one user.

Foreign Keys:

Bookings contains `email` (FK) as a foreign key, which references the `email` primary key in the `Users` entity. This link establishes which user made a particular booking.

User Attributes: The `Users` entity has the following attributes:

`email`: The primary key and unique identifier for the user.

`name`: The name of the user.

`password`: The hashed or encrypted password for user authentication.

`logins`: Likely represents the number of times a user has logged in, or related login metadata.

Booking Attributes: The `Bookings` entity captures comprehensive details about each reservation, including:

`booking_id`: The primary key for the booking.

`email` (FK): The foreign key linking the booking to a specific user.

`type`: Specifies the type of booking (e.g., bus, train, flight, hotel) as described in the project.

`source`: The origin location for travel bookings.

destination: The destination location for travel bookings.

date: The date of the travel or accommodation.

seat: The specific seat number (for bus/train/flight) or room details (for hotel).

details: A general field for additional booking-specific information (e.g., hotel name, flight number).

price: The total cost of the booking.

payment_method: The method used for payment (e.g., credit card, UPI).

payment_reference: A unique identifier for the payment transaction.

Cardinality:

As per the relationship, one user can initiate multiple bookings, but each booking record is explicitly tied back to a single user.

Normalization and Structure:

The ER diagram exhibits good normalization principles. Data is logically separated into **Users** and **Bookings** entities, minimizing redundancy and enhancing data integrity. The use of primary and foreign keys correctly establishes relationships.

Use Case Coverage: This ER model effectively supports the core functionalities of the TravelGo platform as described:

User registration and login (**Users** entity).

Storing and managing various types of travel and accommodation bookings (**Bookings** entity with **type**, **source**, **destination**, **date**, **seat**, **details**).

Tracking booking costs and payment details (**price**, **payment_method**, **payment_reference**).

Linking bookings directly to the user who made them.

Scalability and Cloud Relevance:

The simple yet effective two-entity structure is well-suited for a cloud-based solution like TravelGo, which uses DynamoDB. The **email** as PK for **Users** and **booking_id** for **Bookings** provides clear partition keys, which aligns well with NoSQL database design principles for efficient storage and retrieval of user and booking data.

Description

Service / Tool	Documentation Link
AWS Account Setup	https://docs.aws.amazon.com/accounts/latest/reference/getting-started.html
AWS IAM (Identity and Access Management)	https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html
AWS EC2 (Elastic Compute Cloud)	https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html
AWS DynamoDB	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html
Amazon SNS (Simple Notification Service)	https://docs.aws.amazon.com/sns/latest/dg/welcome.html
Git Documentation	https://git-scm.com/doc
Visual Studio Code Installation	https://code.visualstudio.com/download

Epic 1: Backend Development and Application Setup

Story 1: Develop the Backend Using Flask

Story 2: Integrate AWS Services Using boto3

Epic 2: AWS Account Setup and Login

Story 1: Set Up an AWS Account

Story 2: Log In to the AWS Management Console

Epic 3: DynamoDB Database Creation and Setup

Story 1: Create a DynamoDB Table

Story 2: Configure Attributes for User Data and Book Requests

Epic 4: SNS Notification Setup

Story 1: Create SNS Topics for Book Request Notifications

Story 2: Subscribe Users and Library Staff to SNS Email Notifications

Epic 5: IAM Role Setup

Story 1: Create IAM Role

Story 2: Attach Policies

Epic 6: EC2 Instance Setup

Story 1: Launch an EC2 Instance

Story 2: Configure Security Groups for HTTP and SSH Access

Epic 7: Deployment on EC2

Story 1: Upload Flask Files

Story 2: Run the Flask Application

Epic 8: Testing and Deployment

Story 1: Conduct Functional Testing

Conclusion

Description

This is for your understanding only, please refrain from creating an AWS account. A temporary account will be provided via Troven.

Go to the AWS website (<https://aws.amazon.com/>).

Click on the "Create an AWS Account" button.

Follow the prompts to enter your email address and choose a password.

Provide the required account information, including your name, address, and phone number.

Enter your payment information. (Note: While AWS offers a free tier, a credit card or debit card is required for verification.)

Complete the identity verification process.

Choose a support plan (the basic plan is free and sufficient for starting).

Once verified, you can sign in to your new AWS accounts

Log in to the AWS Management Console

After setting up your account, log in to the [AWS Management Console](#).